
专题：通过软件模拟构建自己的虚拟机

接下来的几题将暂时将重点从 Java 编程转移到剖析计算机的内部结构。我们将介绍机器语言编程，并编写几个机器语言程序。为了使这一体验更有价值，我们将通过软件模拟（仿真）技术构建一个名为 Simpletron 的虚拟机，你可以在它上面执行自己编写的机器语言程序。

6.19（机器语言编程）让我们创建一个名为 Simpletron 的虚拟机。顾名思义，它是一台结构简单但功能强大的机器。Simpletron 只能运行用其唯一能直接理解的语言——Simpletron 机器语言（Simpletron Machine Language, SML）——编写的程序。

Simpletron 包含一个**累加器**（accumulator），这是一个特殊的寄存器，用于在进行计算或检查之前存放信息。Simpletron 中的所有信息都以**字**（word）为单位处理。一个字是一个带符号的 4 位十进制数，例如+3364、-1293、+0007 和-0001。Simpletron 配备了 100 个字的内存，这些字通过其内存地址 00、01、……、99 来引用。

在运行 SML 程序前，首先必须将程序加载到内存中。每个 SML 程序的第一个指令（或语句）都放在地址 00 处，模拟器（仿真器）将从该位置开始执行。

用 SML 编写的每个指令都占用 Simpletron 内存中的一个字（因此指令本身也是带符号的 4 位十进制数）。我们假定 SML 指令字的符号始终为正，但数据字的符号可正可负。每个 Simpletron 内存地址都可能包含一个指令或供程序使用的数据值，某些内存地址也可能未使用。每个 SML 指令的前两位数字是操作码，用于指定要执行的操作。下面总结了 SML 操作码。

操作码	含义
输入/输出操作:	
final int READ = 10;	从用户读取一个字到内存中的特定地址
final int WRITE = 11;	将特定内存地址中的字写入屏幕
加载/存储操作:	
final int LOAD = 20;	将特定内存地址中的字加载到累加器中
final int STORE = 21;	将累加器中的字存储到特定内存地址
算术运算:	

<code>final int ADD = 30;</code>	将特定内存地址中的字与累加器中的字相加（结果保留在累加器中）
<code>final int SUBTRACT = 31;</code>	从累加器中的字减去特定内存地址中的字（结果保留在累加器中）
<code>final int DIVIDE = 32;</code>	用特定内存地址中的字除 ^① 累加器中的字（结果保留在累加器中）
<code>final int MULTIPLY = 33;</code>	将累加器中的字乘以特定内存地址中的字（结果保留在累加器中）
<i>控制转移操作：</i>	
<code>final int BRANCH = 40;</code>	跳转到特定的内存地址
<code>final int BRANCHNEG = 41;</code>	如果累加器为负，则跳转到特定的内存地址
<code>final int BRANCHZERO = 42;</code>	如果累加器为零，则跳转到特定的内存地址
<code>final int HALT = 43;</code>	停止（Halt），表示程序已完成其任务

SML 指令的后两位数字是操作数，即操作所应用的字所在的内存地址。我们来看几个简单的 SML 程序。

SML 加法程序

以下 SML 程序从用户处读取两个数字，计算并显示两者之和。

地址	数字	指令
00	+1007	（读取 A）
01	+1008	（读取 B）
02	+2007	（加载 A）
03	+3008	（加 B）

^① 译注：注意是除，不是除以。

04	+2109	(存储 C)
05	+1109	(写入 C)
06	+4300	(停止)
07	+0000	(变量 A)
08	+0000	(变量 B)
09	+0000	(结果 C)

该程序的操作如下：

- 指令+1007 从用户处读取第一个数字，并将其放入地址 07（已初始化为 0）；
- +1008 将下一个数字读入地址 08；
- 加载指令+2007 将第一个数字放入累加器；
- 加法指令+3008 将第二个数字与累加器中的数字相加，所有 SML 算术指令都将其结果保留在累加器中；
- 存储指令+2109 将结果放入内存地址 09；
- 写入指令+1109 从内存地址 09 获取数字并显示它（作为带符号的 4 位十进制数）；
- 停止指令+4300 终止程序执行。

SML 比较程序

以下 SML 程序从用户处读取两个数字，并判断和显示较大的值。指令+4107 是一次条件控制转移，类似于 Java 的 if 语句。

地址	数字	指令
00	+1009	(读取 A)
01	+1010	(读取 B)
02	+2009	(加载 A)
03	+3110	(减 B)
04	+4107	(如果为负则跳转到 07)
05	+1109	(写入 A)
06	+4300	(停止)
07	+1110	(写入 B)

08	+4300	(停止)
09	+0000	(变量 A)
10	+0000	(变量 B)

现在，请编写 SML 代码来完成以下各项任务：^①

- 使用哨兵控制的循环连续读取正数，计算并显示其总和；
- 使用计数器控制的循环读取 7 个正数和负数，计算并显示它们的平均值；
- 读取一系列数字，并判断和显示其中最大的数字。读取的第一个数字指出程序应处理多少个数字。

6.20 (Simpletron 模拟器) 本题要求构建自己的虚拟机。你将使用强大的软件模拟技术为练习 6.19 描述的 Simpletron 创建一个模拟器。该 Simpletron 模拟器会将你的计算机变成一台 Simpletron，以便运行、测试和调试你在练习 6.19 中编写的 SML 程序。当运行 Simpletron 模拟器时，它应该显示：

```
*** 欢迎来到 Simpletron! ***
*** 请一次输入一条程序指令 ***
*** (或数据字)。我将显示 ***
*** 内存地址和一个问号 (?)。 ***
*** 然后，您输入该地址的字。 ***
*** 输入-99999 结束程序输入。 ***
```

程序应使用一个包含 100 个元素的一维数组 `memory` 来模拟 Simpletron 的内存。现在假设模拟器正在运行，我们来看一下输入练习 6.19 中第二个 SML 程序时的交互过程。

```
00 ? +1009
01 ? +1010
02 ? +2009
03 ? +3110
04 ? +4107
05 ? +1109
06 ? +4300
07 ? +1110
08 ? +4300
09 ? +0000
10 ? +0000
11 ? -99999
```

程序应显示内存地址，后跟一个问号。用户在上述交互过程中的问号右侧输入每个值。当

^① 译注：以下各题的参考答案分别是 `sum.txt`、`average.txt` 和 `max.txt`。稍后会解释如何用 Java 程序加载并运行这些 SML 代码。

用户输入哨兵值-99999 时，程序应显示：

```
*** 程序加载完成 ***
*** 程序开始执行 ***
```

SML 程序此时已被放置（或加载）到数组 `memory` 中。现在，Simpletron 开始执行 SML 程序。执行从地址 00 的指令开始，并且像在 Java 中一样，按顺序进行，除非遇到控制转移指令。

使用变量 `accumulator` 表示累加器寄存器，`instructionCounter` 跟踪正在执行的指令所在的内存地址，`operationCode`（操作码）表示当前的操作（即指令字的左两位数字），`operand`（操作数）表示当前指令操作的内存地址（即指令字的右两位数字）。不要直接从内存执行指令，而应先将下一条指令从内存传输到 `instructionRegister`，然后从中提取左两位数字放入 `operationCode`，右两位数字放入 `operand`。当 Simpletron 开始执行时，所有特殊寄存器都初始化为零。

我们来逐步执行程序的第一个 SML 指令，即内存地址 00 中的+1009。这个过程被称为**指令执行周期**（instruction-execution cycle）。`instructionCounter` 告诉我们要执行的下一条指令的地址。我们使用以下 Java 语句从内存中获取该地址的内容：

```
instructionRegister = memory[instructionCounter];
```

操作码和操作数通过以下语句从指令寄存器中提取：

```
operationCode = instructionRegister / 100;
operand = instructionRegister % 100;
```

现在，Simpletron 必须确定操作码是一次读取操作（而不是写入、加载等）。我们用一个 `switch` 语句或 `switch` 表达式来区分 12 个 SML 操作。下面展示了如何在 `switch` 中模拟各种 SML 指令的行为。我们稍后会讨论分支指令，其他指令则留给你自己完成。

指令	描述
读取:	显示提示消息“请输入一个整数”，读取用户输入的整数，并将其存储到地址 <code>memory[operand]</code> 中
加载:	<code>accumulator = memory[operand];</code>
加:	<code>accumulator += memory[operand];</code>
停止:	终止 SML 程序的执行并显示“*** Simpletron 终止执行 ***”

当 SML 程序完成执行时，应显示每个寄存器的名称和内容以及完整的内存内容。这种输出通常称为**计算机转储**（computer dump）。为了帮助你编写 `dump` 方法，以下是一个示例转储格式。

```
寄存器:
accumulator  +0000
instructionCounter 00
```

```

instructionRegister    +0000
operationCode 00
operand  00
内存:
      0      1      2      3      4      5      6      7      8      9
0  +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
10 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
20 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
30 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
40 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
50 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
60 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
70 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
80 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
90 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000

```

执行 Simpletron 程序后，将通过上述转储来显示当执行终止时寄存器和内存中的值。

我们继续执行程序的第一个指令——地址 00 中的+1009。**switch** 通过提示用户输入一个值，读取该值并将其存储在内存地址 `memory[operand]` 中来模拟此任务。该值被读入地址 09。

此时，第一个指令的模拟完成。剩下的就是准备 Simpletron 以执行下一个指令。由于刚执行的指令不是控制转移，因此我们递增指令计数器寄存器，如下所示：

```
++instructionCounter;
```

这样便完成了第一个指令的模拟执行。整个过程（即指令执行周期）从获取要执行的下一个指令重新开始。

现在，我们来考虑 Simpletron 如何模拟分支指令——即控制转移。我们只需适当地调整指令计数器中的值即可。因此，无条件分支指令（40）在 **switch** 中模拟为：

```
instructionCounter = operand;
```

类似地，“如果累加器为零则分支”条件指令模拟为：

```
if (accumulator == 0) { instructionCounter = operand; }。
```

现在，请实现自己的 Simpletron 模拟器并运行在练习 6.19 中编写的每个 SML 程序。如果愿意，可以为 SML 添加其他功能并在模拟器中提供这些功能。

该模拟器应检查各种类型的错误。例如，在程序加载阶段，用户输入到 Simpletron 内存中的每个数字都必须在 -9999~+9999 的范围内。模拟器应测试输入的每个数字是否在此范围内，如果不是，则应继续提示用户重新输入，直到他们输入正确的数字。

在执行阶段，模拟器应检查各种严重错误，例如尝试除以零、尝试执行无效操作码以及累加器溢出（算术运算产生超出 -9999~+9999 范围的值）。此类严重错误被称为**致命错误**（fatal error）。当检测到致命错误时，模拟器应显示一条错误消息，例如：

```

*** 尝试除以零 ***
*** Simpletron 执行异常终止 ***

```

而且应该以前述的格式显示完整的计算机转储。这种处理将帮助用户找到程序中的错误。

专题：系统软件案例学习——构建你自己的编译器

在第 6 章中，我们介绍了虚构的 Simpletron 机器语言（SML）。在练习 6.20 中，你通过软件模拟创建了一个可执行 SML 程序的 Simpletron 虚拟机。在这个颇具挑战性的“专题”中，你将构建一个“玩具”编译器，它能将用 Simple（一种虚构的、简洁的高级编程语言）编写的程序转换为 SML。本节将编程过程中的关键环节有机地结合起来，你将完成任务以下任务：

- 编写几个 Simple 高级语言程序；
- 使用自己构建的编译器来编译这些程序，并将生成的 SML 机器语言代码输出到文件；
- 从该文件将 SML 代码加载到 Simpletron 的内存中；
- 在你于练习 6.20 中构建的 Simpletron 虚拟机上执行这些 SML 程序。

本专题包含总共 6 个练习。前两个涵盖了实现编译器所需的重要计算机科学技术。第三个练习介绍 Simple 高级语言，提供了一些完整的代码示例，并要求你编写几个自己的 Simple 程序。第 4 个练习指导你构建 Simple 编译器。第 5 个练习介绍编译器优化这一关键主题，你将修改编译器以减少其生成的 SML 指令数量，从而使 SML 程序更节省内存、执行速度更快。最后一个练习则要求通过修改编译器来添加更多功能。

12.22（中缀转后缀程序）编译器通常使用栈（stack）来帮助进行表达式求值和机器语言代码生成。Java 的 Stack 泛型类提供了几个相关的方法。例如，empty 方法返回一个布尔值以判断栈是否为空；peek 方法返回对栈顶元素的引用但不移除该元素；push 方法接收一个元素并将其置于栈顶；pop 方法则移除栈顶元素并返回其引用。

在本练习及后续练习中，我们将探讨编译器如何对仅由 1 位整数常量、操作符和圆括号组成的算术表达式进行求值。你可以轻松地修改我们提供的算法，使其同样适用于多位整数和浮点数。

① 译注：本书中文版配套资源提供了示例程序 Simpletron.java，它既支持手动输入代码，也支持从一个 SML 代码文件中加载。例如，可以运行 `java Simpletron.java sum.txt` 来完成 6.19a 那道题，即连续输入多个要求和的正数，任意负数将结束输入并在内存 dump 的地址 98 处显示结果。同理，可以运行 `java Simpletron.java max.txt` 来完成 6.19c 那道题，并连续输入 3、44、88、67，随后会在内存 dump 的地址 98 处显示最大值。详情请访问 <https://bookzhou.com>。另外，还提供了中英文版线上 Simpletron 模拟程序，网址是 <https://simpletron.bookzhou.com>，开源地址是 <https://github.com/transbot/Simpletron>。

人们经常编写如 $3 + 4$ 和 $7 / 9$ 这种将操作符置于操作数之间的表达式，这称为**中缀表示法**或**中缀记法**（infix notation）。而计算机更偏爱**后缀表示法**或**后缀记法**（postfix notation），即将操作符写在其两个操作数的右侧。上述中缀表达式对应的后缀表示法为 $3\ 4\ +$ 和 $7\ 9\ /$ 。

为了对中缀表达式进行求值，某些编译器会执行以下操作：

- 首先将表达式转换为后缀表示法；
- 然后求值其后缀版本。

上述两个基于栈的算法都只需要对表达式进行一次从左到右的遍历。在本练习中，你将实现**中缀转后缀算法**；在下一个练习中，则将实现**后缀表达式求值算法**。

请编写一个程序，将包含 1 位整数的有效中缀算术表达式（例如 $(6 + 2) * 5 - 8 / 4$ ）转换为后缀表达式。该中缀表达式的后缀版本为 $6\ 2\ +\ 5\ *\ 8\ 4\ /\ -$ 。请注意，后缀表达式不包含圆括号。程序应将中缀表达式读入一个名为 `infix` 的 `String`，并将转换后的后缀表达式存入一个名为 `postfix` 的 `String`。

使用 `Stack<Character>` 创建后缀表达式的算法如下：首先，将一个左圆括号 '(' 压入栈，并在 `infix` 字符串的末尾追加一个右圆括号 ')'。然后，当栈不为空时，从左到右读取 `infix` 字符串中的字符，并执行相应操作：^①

- 如果当前字符是数字，则将其追加到 `postfix`；
- 如果当前字符是左圆括号，则将其压入栈；
- 如果当前字符是操作符，则当栈顶存在优先级等于或高于当前操作符的其他操作符时，将它们逐个弹出并追加到 `postfix`，最后将 `infix` 中的当前字符压入栈；
- 如果当前字符是右圆括号，则从栈顶弹出操作符并追加到 `postfix`，直到栈顶是左圆括号为止，最后从栈中弹出并丢弃该左圆括号。

表达式中允许使用的算术运算包括+（加）、-（减）、*（乘）和/（除）。在本练习中，建议每次修改栈时都显示其字符串表示，以确认算法实现是否正确。你的解决方案应包含一个 `main` 方法以及以下几个方法：一个名为 `convertToPostfix` 的方法，它接收代表中缀表达式的 `String` 并返回其等效的后缀表达式 `String`；一个名为 `isOperator` 的方法，它返回一个布尔值以判断其 `char` 参数是否为操作符；以及一个名为 `precedence` 的方法，它返回一个布尔值以判断其第一个 `char` 参数（`operator1`）的优先级是否大于或等于第二个 `char`

^① 译注：这就是经典的“中缀转后缀（逆波兰）”算法。

参数 (operator2) 的优先级。^①

12.23 (后缀表达式求值程序) 编写一个程序, 用于对一个有效的后缀表达式 (例如 `6 2 + 5 * 8 4 / -`) 进行求值。程序应读取一个代表后缀表达式的 `String`, 该表达式仅包含 1 位整数操作数和二元操作符, 且不含圆括号 (如前所述, 圆括号已在“中缀转后缀”的过程中消除了)。程序应扫描该后缀表达式, 并使用以下算法对其进行求值:^②

1. 只要未到达后缀表达式 `String` 的末尾, 就从左到右读取该表达式。如果当前字符是数字, 则将其对应的整数值压入栈 (要获取一个数字字符的整数值, 可将其在字符集中的值减去字符 `'0'` 在字符集中的值)。否则, 如果当前字符是操作符, 则将栈顶两个元素依次弹出并分别存入变量 `x` 和 `y` 中, 计算“`y 操作符 x`”的值, 然后将结果压回栈。^③
2. 一旦到达后缀表达式 `String` 的末尾, 就弹出栈顶的值。该值即为后缀表达式的最终结果。

此算法仅支持二元算术操作符。例如, 在步骤 1 中, 如果遇到的操作符是 `'/'`, 且此时栈顶元素是 2、次顶元素是 8, 那么程序会将 2 弹出并存储到 `x` 中, 将 8 弹出并存储到 `y` 中, 然后求值 `8 / 2`, 并将结果 4 压回栈中。此规则适用于所有二元算术操作符。在本练习中, 请使用 `Stack<Integer>`。表达式允许使用的算术运算包括 `+` (加)、`-` (减)、`*` (乘) 和 `/` (除)。你的解决方案应包含一个 `main` 方法以及以下几个方法: `evaluatePostfixExpression` 方法接收一个代表后缀表达式的 `String`, 对其进行求值, 然后返回 `int` 类型的结果; `calculate` 方法接收两个 `int` 类型的操作数和一个 `char` 类型的操作符, 用于求值“`operand1 操作符 operand2`”这个表达式, 并返回其 `int` 类型的结果。

12.24 (Simple 编程语言和 Simple 程序) 在构建实际的编译器之前, 我们先来讨论一种类似于早期 BASIC 的、简单而功能强大的高级编程语言, 我们称之为 Simple。每个 Simple 语句都由一个行号和一个指令构成, 且行号必须按升序排列。如下所示, 每个指令都以 `rem`^④、`input`、`let`、`print`、`goto`、`if...goto` 或 `end` 等 Simple 命令之一开头。

^① 译注: 和上个例子相反, 本例使用的栈只存储操作数, 而不存储操作符和左右圆括号。另外, 由于要求只处理 1 位整数 (不允许负数), 所以支持在输入中包含和不包含空格这两种情况。例如, 无论输入 `321-*2+` 还是 `3 21-* 2+`, 都会转换为原始中缀表达式 `3*(2-1)+2`, 并输出结果 5。本书中文版配套资源提供了示例程序 `PostfixEvaluator.java`, 详情请访问 <https://bookzhou.com>。

^② 译注: 这就是经典的“中缀转后缀 (逆波兰)”算法。

^③ 译注: 换言之, 先弹出的是右操作数, 后弹出的是左操作数。

^④ 译注: 这里一比一还原了 BASIC 语言, `rem` 是 `remark` 的简称, 代表“注释”。

命令	示例语句	描述
rem	50 rem this is a remark	rem 命令后的文本是注释，将被编译器忽略，不会生成 SML 代码
input	30 input x	显示一个问号以提示用户输入一个整数，然后从键盘读取该整数并将其存入变量 x
let	80 let u = 4 * (j - 7)	将表达式 4 * (j - 7) 的值赋给变量 u。等号右侧可以是任意复杂的中缀表达式（和之前的练习一样，暂时只允许 1 位整数、左右圆括号和二元操作符）
print	10 print w	显示变量 w 的值
goto	70 goto 45	将程序控制权转移到第 45 行的语句
if...goto	35 if i == z goto 80	比较 i 和 z 的值是否相等，如果条件为真，则将控制权转移到第 80 行的语句；否则，继续执行下一个语句
end	99 end	终止程序执行

Simple 仅允许使用 +、-、* 和 / 操作符来对整数表达式求值，这些操作符的优先级与 Java 中的相同。圆括号可以改变表达式的求值顺序。练习 12.27 将提出一些对 Simple 编译器进行增强的建议，其中部分功能（例如增加浮点数处理能力）将需要修改 Simpletron 虚拟机。

其他 Simple 语言规则

Simple 语言还具有以下规则：编译器仅识别小写字母，因此 Simple 程序中的所有字符都应小写；变量名是单个字母，不允许使用多字符变量名，因此程序应在 rem 语句中对变量的用途加以说明；仅使用 int 类型的变量；没有变量声明，一旦在程序中首次提及某个变量名，即完成了该变量的声明，并将其初始化为零；语法不支持字符串操作（如读取、写入或比较字符串等）；使用条件分支语句 if...goto 和无条件分支语句 goto 来改变程序的控制流，当 if...goto 语句中的条件为真时，控制权将转移到指定的行号，该语句中可以使用关系操作符（<、>、<=、>=）和相等性操作符（==、!=），它们的优先级与 Java 中的相同。

编译器假定输入的 Simple 程序是正确的。练习 12.27 会要求增强编译器以检查语法错误。

简单程序示例

我们来看几个演示 Simple 语言特性的程序。下面这个程序会从键盘读取两个整数，将它们分别存入变量 **a** 和 **b**，然后计算并打印两者之和（存入变量 **c**）。我们加入了空的 **rem** 语句以增强程序的可读性。

```
10 rem 输入两个整数，计算并打印两者之和
15 rem
20 rem 输入两个整数
30 input a
40 input b
45 rem
50 rem 求和并存入 c
60 let c = a + b
65 rem
70 rem 打印结果
80 print c
90 rem 终止执行
99 end
```

```
? 5
? 3
8
```

下面这个程序判断并打印两个整数中较大的一个。程序从键盘读取两个整数并存入变量 **s** 和 **t**，然后通过 **if...goto** 语句测试条件 **s >= t**。如果条件为真，控制权将转移到第 90 行以显示 **s**；否则，程序将显示 **t**，然后将控制权转移到第 99 行的 **end** 语句以终止程序。

```
10 rem 输入两个整数，然后确定并打印较大者
20 input s
30 input t
32 rem
35 rem 测试 s 是否大于或等于 t
40 if s >= t goto 90
45 rem
50 rem t 更大，打印 t
60 print t
70 goto 99
75 rem
80 rem s 更大或相等，打印 s
90 print s
99 end
```

```
? 3
? 7
7
```

```
? 7
? 3
7
```

Simple 语言没有像 Java 的 `for`、`while` 或 `do...while` 那样的迭代语句，但可以使用 `if...goto` 和 `goto` 语句来模拟循环。下面这个程序就使用了一个哨兵控制循环来计算若干整数的平方。程序从键盘读取每个整数并存入变量 `j`。如果输入的是哨兵值 `-9999`，那么控制权将转移到第 99 行，程序随之终止。否则，程序会将 `j` 的平方赋值给 `k`，将 `k` 的值输出到屏幕，然后将控制权交回第 20 行以输入下一个整数。

```
10  rem 计算整数的平方（输入-9999 结束）
20  input j
23  rem
25  rem 检查哨兵值
30  if j == -9999 goto 99
33  rem
35  rem 计算 j 的平方并存入 k
40  let k = j * j
50  print k
53  rem
55  rem 循环，获取下一个 j
60  goto 20
99  end
```

```
? 3
9

? 7
49

? -9999
```

编写自己的 Simple 程序

使用本练习中的 Simple 程序作为指导，编写简单的 Simple 程序来完成以下各项任务：^①

- a) 输入三个整数，求它们的平均值并打印结果；
- b) 使用哨兵控制的循环输入 10 个整数，计算并打印它们之和；
- c) 使用计数器控制的循环输入 7 个整数——一些是正数，一些是负数——计算并打印它们

^① 译注：本书中文版配套资源提供了示例 Simple 语言程序 `12.24.simple`，详情请访问 <https://bookzhou.com>。注意，从 a 到 g 的所有任务均整合到一个程序中。

的平均值；

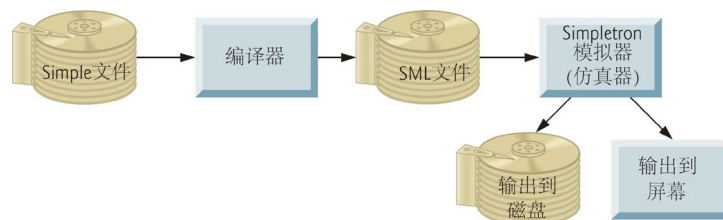
d) 输入一系列整数，判断并打印其中最大的整数。输入的第一个整数指示应处理多少个数字；

e) 输入 10 个整数并打印其中最小的整数；

f) 计算并打印 2~30 的偶整数之和；

g) 计算并打印 1~9 的奇整数之积。

12.25（构建编译器，请先完成练习 6.19、6.20、12.22、12.23 和 12.24）我们已经在练习 12.24 中介绍了 Simple 语言及其程序，接着让我们讨论如何为这种语言构建编译器。下图展示了将 Simple 程序编译成 SML，然后在 Simpletron 虚拟机中执行的过程。



编译器读取 Simple 程序文件，将其编译成 SML 代码，然后以每行一个指令的方式，将 SML 程序写入文本文件。接着，Simpletron 虚拟机将最终的 SML 文件加载到 Simpletron 的 100 个元素的内存数组中，执行程序并将结果输出到屏幕和一个文件。注意，本例会将所有屏幕输出发送到一个文件，以方便打印硬拷贝。

你在练习 6.20 中开发的 Simpletron 虚拟机从键盘获取输入，而不是从文件获取。^①本例需要修改 Simpletron 以从文件读取（使用在第 11 章中学到的文件处理技术），以便它可以运行由 Simple 编译器生成的程序。

编译器对 Simple 程序执行两次扫描以将其转换为 SML，如下所示：

- 第一次扫描构建一个**符号表**（稍后讨论）。编译器将 Simple 程序的每个行号、变量名和常量存储到符号表中。每个都与其类型及其在最终 SML 代码中的位置一起存储。第一次扫描还生成与每个 Simple 语句对应的 SML 指令。如果 Simple 程序包含将控制权转移到程序后面行的语句，则第一次扫描将生成包含一些不完整指令的 SML 程序。

^① 译注：本书中文版的示例程序已经实现了从文本文件中读取输入并运行的 Simpletron 虚拟机，代码文件是 Simpletron.java。

-
- 第二次扫描定位并完成未完成的指令，并将 SML 程序输出到文件。

第一次扫描

编译器首先将 Simple 程序的第一个语句读入内存。编译器将该行分解为单独的词元（即语句的“片段”）以进行处理和编译。可以使用 7.6 节介绍的 `String` 方法来确定语句的词元。如前所述，每个语句都以行号开头，后跟一个命令。当编译器将语句的剩余部分分解为词元时，如果词元是行号、变量或常量，则将其放置到符号表中。目前，只有当行号是语句中的第一个词元时，才将其放置到符号表中——稍后将讨论编译器如何处理作为条件或无条件分支目标的行号。

`symbolTable` 对象是一个 `ArrayList<TableEntry>`，表示程序中的每个符号。程序中的符号数量没有限制，因此 `symbolTable` 可能很大——目前暂时把它设置为 200 个元素。一旦有了一个可以工作的编译器，就可以调整它的大小。

`TableEntry` 可以声明为记录类型：

```
record TableEntry(int symbol, char type, int location) {};
```

每个 `TableEntry` 包含如下所示的三个成员：

- `symbol` 是一个整数，包含变量的字符表示形式（变量名同样是单个字符）、行号或整数常量。我们知道，像 'x' 这样的字符在底层是用一个整数值来表示的。
- `type` 是一个字符，用于指示符号的类型——'C' 表示常量，'L' 表示行号，'V' 表示变量。
- `location` 包含与符号关联的 Simpletron 内存位置（00~99）。Simpletron 内存是一个包含 100 个整数的数组，用于存储 SML 指令和数据。如果 `location` 关联的是行号，那么它对应的是 Simpletron 内存数组中的一个元素，Simple 语句的 SML 指令将从该元素开始。如果 `location` 关联的是变量或常量，那么变量或常量就存储在与该 `location` 对应的 Simpletron 内存数组元素中。变量和常量从 Simpletron 内存位置 99 向下分配。第一个变量或常量存储在位置 99 中，下一个存储在位置 98 中，以此类推。

符号表

符号表在将 Simple 程序转换为 SML 中起着不可或缺的作用。我们在练习 6.19 中了解到，SML 指令是一个带符号的 4 位整数，其中包括**操作码**和**操作数**。操作码由 Simple 命令确定。例如，Simple 命令 `input` 对应于 SML 操作码 10（SML READ 命令），Simple 命令 `print` 对应于 SML 操作码 11（SML WRITE 命令）。操作数是一个内存位置，包含操作码在其上执行任务的数据。例如，操作码 10 从键盘读取一个值，并将其存储到操作数指定的内存位置中。编译器搜索 `symbolTable` 以确定每个符号的 Simpletron 内存位置，以便可以使用相应的 `location` 来完成 SML 指令。

将 Simple 程序编译成 SML

每个 Simple 语句的编译过程都基于特定的命令。例如，在 `rem` 语句中的行号插入到符号表中之后，编译器会忽略该语句的其余部分——`rem` 语句仅用于注释——不生成 SML 代码。`input`、`print`、`goto` 和 `end` 语句分别对应于 SML `READ`、`WRITE`、`BRANCH`（跳转到特定位置）和 `HALT`（终止程序）指令。编译器将包含这些 Simple 命令的语句直接转换为 SML。如果指定的行号引用了 Simple 程序文件中后面的语句，则 `goto` 语句最初可能包含未解析的引用。这称为**前向引用**（forward reference）。

编译 goto 语句

当编译带有未解析引用的 `goto` 语句时，必须标记 SML 指令以指示编译器的第二次扫描必须完成该指令。这些标志存储在一个名为 `flags` 的 100 个元素的 `int` 数组中，其中每个元素都初始化为 -1。如果行号引用的内存位置尚不知道（换言之，它不在符号表中），就将其行号存储到数组 `flags` 中，该元素与不完整指令具有相同的数组下标。不完整指令的操作数暂时设置为 00。例如，无条件分支指令（进行前向引用）保留为 +4000，直到编译器的第二次扫描为止，我们稍后将对此进行解释。

编译 if...goto 语句

编译 `if...goto` 或 `let` 语句比其他语句更复杂——每个语句都会生成多个 SML 指令。对于 `if...goto` 语句，编译器会生成代码来测试条件并可能分支到另一行。分支的结果可能是未解析的前向引用。可以使用 SML 的 `BRANCHZERO` 和 `BRANCHNEG` 指令（或两者的组合）^①来模拟每个 Simple 关系和相等性操作符。

编译 let 语句

对于 `let` 语句，编译器会生成代码对任意复杂的中缀算术表达式进行求值。这种表达式由操作符、单字母整数变量名、整数常量以及可能的圆括号组成。表达式应该使用空格来分隔每个操作数和操作符。练习 12.22 和 12.23 介绍了编译器的中缀到后缀转换算法以及后缀表达式求值算法。在构建编译器之前，请先完成这些练习。编译器将每个表达式从中缀表示法转换为后缀表示法，再对后缀表达式进行求值。编译器在求值后缀表达式时生成 SML 机器语言指令。

编译表达式

那么，编译器如何生成机器语言来对包含变量的表达式进行求值？后缀表达式求值算法包含一个“钩子”（hook），允许我们的编译器生成 SML 指令，而不是实际计算表达式的结

^① `BRANCHZERO` 指令在累加器为零时跳转到特定的内存地址。`BRANCHNEG` 指令在累加器为负时跳转到特定的内存地址。详情请参见练习 6.19。

果。要在编译器中启用此“钩子”，必须修改后缀表达式求值算法以执行以下任务：

- 在符号表中查找它遇到的每个符号（并可能插入它）；
- 确定符号相应的内存位置；
- 将内存位置而不是符号压入栈。

当在后缀表达式中遇到操作符时，栈顶的两个内存位置被弹出，并使用内存位置作为操作数来生成用于实现操作的 SML。每个子表达式的结果都存储在一个临时内存位置中，并被压回栈，以便可以继续进行后缀表达式的求值。当后缀表达式求值完毕时，结果的内存位置是栈上唯一剩下的位置。我们弹出这个位置，并生成 SML 指令以将结果赋给 `let` 语句左侧的变量。

第二次扫描

编译器的第二次扫描执行以下两个任务：

- 解析任何未解析的引用；
- 将 SML 代码输出到文件。

每个引用的解析过程如下：

1. 在 `flags` 数组中查找未解析的引用（即值不为 -1 的元素）；
2. 根据 `flags` 数组中存储的符号，在 `symbolTable` 中找到对应的元素（并确保其类型为 'L'，代表“行号”）；
3. 将该符号的 `location` 成员所记录的内存地址，插入到原先操作数为 00 的、含有未解析引用的指令中；
4. 重复以上步骤，直到处理完 `flags` 数组中的所有元素。

在解析过程完成后，编译器会将 SML 代码输出到文件（每行一个指令）。Simpletron 可以读取此文件并执行其中的指令（在修改模拟器以支持从文件读取输入之后）。^①

一个完整的例子

我们来讨论一个将 Simple 程序完整转换为 SML 的例子。这个 Simple 程序的功能是：接收用户输入的一个整数，计算从 1 到该整数所有整数之和，并打印结果。例如，如果用户输入 4，那么程序将计算 $1 + 2 + 3 + 4$ 并打印结果 10。图 12.17 展示了该程序的代码以及编译

^① 译注：如前所述，本书中文版在练习 6.19～练习 6.20 中开发的示例程序 Simpletron.java 已支持从文件中读取。

器第一次扫描后生成的SML指令。图12.18是编译器第一次扫描后构造的符号表。图12.19则表明编译器从内存位置99开始向下分配Simpletron的内存。稍后，我们将解释编译器是如何创建这些表的。

Simple 程序	SML 位置和指令	描述
5 rem 输入 x	无	rem 被忽略
10 input x	00 +1099	将 x 读入位置 99
15 rem 检查 y == x	无	rem 被忽略
20 if y == x goto 60	01 +2098 02 +3199 03 +4200	将 y（位置 98）加载到累加器 将 x（位置 99）从累加器上减去 为零则跳转到未解析的位置
25 rem 递增 y	无	rem 被忽略
30 let y = y + 1	04 +2098 05 +3097 06 +2196 07 +2096 08 +2198	将 y（位置 98）加载到累加器 将 1（位置 97）加到累加器上 存储到临时位置 96 从临时位置 96 加载 将累加器存储到 y 中（位置 98）
35 rem 在 total 上累加 y	无	rem 被忽略
40 let t = t + y	09 +2095 10 +3098 11 +2194 12 +2094 13 +2195	将 t（位置 95）加载到累加器 在累加器上加 y（位置 98） 存储到临时位置 94 从临时位置 94 加载 将累加器存储到 t 中（位置 95）
45 rem 重复 y == x 测试	无	rem 被忽略

50 goto 20	14 +4001	跳转到位置 01
55 rem 输出结果	无	rem 被忽略
60 print t	15 +1195	向屏幕输出 t（位置 95）
99 end	16 +4300	终止执行

图 12.17 编译器第一遍扫描后生成的 SML 指令

符号	类型	位置
5	L	00
10	L	00
'x'	V	99
15	L	01
20	L	01
'y'	V	98
25	L	04
30	L	04
1	C	97
分配临时位置 96		
35	L	09
40	L	09
't'	V	95
分配临时位置 94		
45	L	14
50	L	14
55	L	15

60	L	15
99	L	16

图 12.18 为图 12.17 的程序构造的符号表

数据计数器	值	类型
...		
93		下一个要分配的 Simpletron 内存位置
94	无	临时变量
95	't'	变量
96	无	临时变量
97	1	常量
98	'y'	变量
99	'x'	变量

图 12.19 编译器从内存的最后一个位置（99）向下分配 Simpletron 内存

在查看 Simple 程序及其对应的 SML 指令时，请记住 Simple 语言没有变量声明，且 Simpletron 模拟器会将所有内存位置都初始化为零。因此，当编译器第一次遇到某个变量时，会为其创建变量；如果该变量未被初始化，其默认值将为零。

大多数 Simple 语句都能直接转换为单个 SML 指令，但上述程序中的 rem 语句、第 20 行的 if...goto 语句以及第 30 行和第 40 行的 let 语句是例外。注释（rem）本身不会被转换为机器语言，但其行号仍会被存入符号表，以备 goto 或 if...goto 语句引用——Simple 语言允许跳转到注释行。

程序第 20 行指定，如果条件 $y == x$ 为真，那么程序控制权应转移到第 60 行。由于第 60 行出现在程序的后续部分，因此编译器在第一次扫描时无法将其放入符号表——行号只有在作为编译器已处理语句的第一个词元出现时才会被存入符号表。因此，编译器此时无法确定 SML 指令数组中位置 03 处的 SML 分支指令（BRANCHZERO）的操作数。编译器会将行

号 60 存入 `flags` 数组的位置 03，以此标记该指令需要由第二次扫描来完成。

我们必须持续跟踪 SML 数组中的下一个可用指令位置，因为 Simple 语句和 SML 指令之间并非一一对应。例如，第 20 行的 `if...goto` 语句会被编译为三个 SML 指令。每生成一个指令，都必须递增指令计数器，使其指向 SML 数组的下一个位置。对于包含许多语句、变量和常量的 Simple 程序而言，Simpletron 有限的内存是一个潜在的问题，编译器可能会耗尽所有可用内存。为了处理这种情况，你的编译器程序应包含一个**数据计数器**，用以跟踪下一个变量或常量将要存入 SML 数组的位置。如果指令计数器的值大于数据计数器的值，则表示 SML 数组已满。此时，编译过程应终止，并且编译器应显示“内存不足”错误消息。

练习 12.27 要求修改 Simpletron 的内存，使其能容纳 1000 个编号为 000~999 的内存单元，这有助于编译更复杂的 Simple 程序。

第一次扫描分步说明

我们来详细解释图 12.17 中 Simple 程序的编译过程。

编译第 5 行

编译器读取程序的第一行：

```
5 rem 输入 x
```

一般使用 `String` 类的 `split` 方法（参见第 7 章）来确定语句中的词元。可以考虑定义自己的方法来将字符串词元化，并返回一个包含这些词元的 `List<String>`。

对于每个语句，第一个词元都是行号——在本例中是“5”。可以使用 `Integer.parseInt` 方法将其转换为 `int`，以便将符号 5 存入符号表。如果符号表中不存在该符号，就将其插入。由于这是程序的第一行，符号表尚不为空，因此我们将 5 作为 'L' 类型（行号）插入，并为其分配 SML 内存数组的第一个位置（00）。尽管这一行是注释，但仍会为行号在符号表中分配空间，以备 `goto` 或 `if...goto` 语句将其作为跳转目标。如果程序跳转到一个 `rem` 语句的行号，控制流将从该 `rem` 语句后的第一个可执行语句继续。`rem` 语句不会生成 SML 指令，因此指令计数器不会递增。

编译第 10 行

然后，编译器对以下语句进行词元化：

```
10 input x
```

行号 10 作为 'L' 类型被存入符号表，并被分配 SML 数组的第一个可用位置（00）——因为程序以一个 `rem` 语句开始，所以指令计数器仍为 00。`input` 命令表明下个词元是一个变

量（只有变量才能作为 `input` 语句的参数）。由于 `input` 直接对应一个 SML 操作码，因此编译器只需确定 `x` 在 SML 数组中的位置。符号 `x` 在符号表中未找到，因此它被作为 'V' 类型（变量）插入，并被分配 SML 数组中的位置 99。数据存储从位置 99 开始并向下分配（98、97 等）。现在可以为该语句生成 SML 代码了。操作码 10（SML 的 `READ` 操作码）乘以 100，然后加上 `x` 的位置（由符号表确定），最终得到指令 +1099。该指令被存储到 SML 数组的位置 00。由于只生成了一个 SML 指令，因此指令计数器递增 1。

编译第 15 行

接着，编译器对以下语句进行词元化：

```
15 rem 检查 y == x
```

编译器在符号表中查找行号 15，未找到，于是将其作为 'L' 类型插入，并分配 SML 数组的下一个位置（01）。同样，`rem` 语句不生成代码，因此指令计数器不递增。

编译第 20 行

接着，编译器对以下语句进行词元化：

```
20 if y == x goto 60
```

行号 20 作为 'L' 类型被插入符号表，并分配 SML 数组的下一个位置 01。`if` 命令表示需要对一个条件进行求值。变量 `y` 在符号表中未找到，因此它被作为 'V' 类型插入，并分配 SML 位置 98。然后，编译器生成对条件进行求值的 SML 指令。SML 中没有与 `if...goto` 直接对应的指令，因此必须使用 `x` 和 `y` 来执行计算，然后根据结果进行分支跳转来模拟。如果 `y` 等于 `x`，那么 `y` 减 `x` 的结果为零。因此，可以结合 SML 的 `BRANCHZERO`（为零则跳转）指令来模拟 `if...goto` 语句。

第一步需要将 `y`（从 SML 位置 98）加载到累加器中，生成指令 01 +2098。然后，从累加器中减去 `x`，生成指令 02 +3199。累加器中的值可能是零、正数或负数。由于操作符是 `==`，因此需要在结果为零时跳转。编译器首先在符号表中查找跳转目标位置（行号 60），但未找到。因此，将 60 存入 `flags` 数组的位置 03，并生成指令 03 +4200。之所以此时无法填写跳转的内存地址，是因为还没有在 SML 数组中为第 60 行分配位置——该地址将在第二次扫描时解析。指令计数器递增到 04。

编译第 25 行

编译器继续处理以下语句：

```
25 rem 递增 y
```

行号 25 作为 'L' 类型被插入符号表，并分配 SML 位置 04。指令计数器不递增。

编译第 30 行

然后对以下语句进行词元化：

```
30 let y = y + 1
```

行号 30 作为'L'类型被插入符号表，并分配 SML 位置 04。let 命令表明这一行是赋值语句。首先，将该行上的所有符号（如果尚不存在）插入符号表。整型常量 1 作为'C'类型插入，并分配 SML 位置 97。接下来，将赋值语句的右侧从中缀表示法转换为后缀表示法，然后对后缀表达式（y 1 +）进行求值。符号 y 在符号表中找到，其对应的内存位置 98 被压入栈。符号 1 也在符号表中找到，其对应的内存位置 97 被压入栈。

当遇到操作符+时，后缀表达式求值算法会先将栈顶元素弹出作为右操作数，再弹出一个元素作为左操作数，然后生成 SML 指令：

```
04 +2098（加载 y）  
05 +3097（加上 1）
```

y 递增后的结果通过指令 06 +2196（存储到临时位置）存储到一个临时内存位置（96）中，然后将该临时位置压入栈。表达式求值完毕后，必须将结果存入 let 语句的变量 y 中。因此，通过以下指令将临时位置的值加载到累加器，再将累加器的值存入 y：

```
07 +2096（加载临时变量）  
08 +2198（存入 y 中）
```

请注意，其中一些 SML 指令——将累加器存入临时位置 96，然后立即从位置 96 重新加载累加器——这显得**冗余**了。消除这种冗余是**编译器优化**的一个例子，我们稍后将对此进行更多说明。

编译第 35 行

当编译器对以下语句进行词元化时：

```
35 rem 在 total 上累加 y
```

它会将行号 35 作为'L'类型插入符号表，并为其分配位置 09。

编译第 40 行

以下语句与第 30 行类似：

```
40 let t = t + y
```

变量 t 作为'V'类型被插入符号表，并分配 SML 位置 95。生成的指令遵循与第 30 行相同的逻辑和格式，指令 09 +2095、10 +3098、11 +2194、12 +2094 和 13 +2195 被相继生成。t + y 的结果在赋给 t（95）之前，被存入了临时位置 94。内存位置 11 和 12 的指令也显得冗余。同样，我们稍后将讨论此优化问题。

编译第 45 行

以下语句是一条注释：

```
45 rem 重复 y == x 测试
```

因此行号 45 作为 'L' 类型被插入符号表，并分配 SML 位置 14。

编译第 50 行

以下语句将控制权转移到第 20 行：

```
50 goto 20
```

行号 50 作为 'L' 类型被插入符号表，并分配 SML 位置 14。SML 中与 goto 等效的是无条件分支（40）指令，它将控制权转移到指定的 SML 位置。编译器在符号表中查找第 20 行，发现它对应 SML 位置 01。于是，操作码（40）乘以 100，再加上位置 01，在位置 14 处生成指令+4001。

编译第 55 行

然后又是一个注释语句，如下所示：

```
55 rem 输出结果
```

因此行号 55 作为 'L' 类型被插入符号表，并分配 SML 位置 15。

编译第 60 行

以下语句是一个输出语句：

```
60 print t
```

行号 60 作为 'L' 类型被插入符号表，并分配 SML 位置 15。SML 中与 print 等效的是操作码 11（WRITE）。编译器从符号表中确定变量 t 的位置，然后将其加到操作码乘以 100 的结果上，于是在位置 15 处生成指令+1195。

编译第 99 行

以下语句是程序的最后一行：

```
99 end
```

行号 99 作为 'L' 类型存储到符号表中，并分配 SML 位置 16。end 命令生成 SML 指令+4300（43 是 SML 的 HALT 指令）。该指令作为最后一个指令写入 SML 内存数组。HALT 指令没有操作数。

第二次扫描小结

在编译器的第二次扫描中，我们在 `flags` 数组中搜索值不为-1 的元素。位置 03 的值为 60，因此编译器知道指令 03 是不完整的。编译器通过在符号表中搜索 60 来完成该指令，确定其内存位置并将其添加到不完整的指令中。本例通过搜索，确定第 60 行对应于 SML 位置 15，因此编译器将位置 03 处的指令从+4200 替换为+4215。至此，Simple 程序已成功编译。

构建自己的编译器

为了构建编译器，必须执行以下各项任务：

1. 首先，修改在练习 6.20 中编写的 Simpletron 虚拟机程序，使其能从用户指定的文件获取输入（参见第 11 章），模拟器还应将其结果以与屏幕输出相同的格式输出到文件。^①
2. 其次，修改练习 12.22 的中缀转后缀算法，以处理多位（而不是 1 位）整数操作数和单字母变量名操作数；可以使用 `String` 类的 `indexOf` 和 `substring` 方法来定位表达式中的常量和变量，并使用 `Integer.parseInt` 方法将常量从字符串转换为整数，同时还要修改后缀表达式的数据表示，以支持变量名和整型常量；^②
3. 接着，修改练习 12.23 的后缀表达式求值算法，以处理多位（而不是 1 位）整数操作数和单字母变量名操作数，该算法现在还应实现之前讨论的“钩子”以生成 SML 指令，而非直接对表达式求值。
4. 最后，构建编译器，整合(2)和(3)部分的功能以对 `let` 语句中的表达式进行求值，程序中应包含执行编译器第一次扫描的方法和执行第二次扫描的方法。

12.26（优化 Simple 编译器）当 Simple 程序被编译并转换为 SML 时，会生成一组指令。某些指令组合会频繁重复出现，通常是以被称为“指令序列”（production）的三元组形式。指令序列一般由三个指令组成，例如 **load**、**add** 和 **store**。以下是编译图 12.17 中的程序时生成的 5 个未优化的 SML 指令：

```
04 +2098 (load)
05 +3097 (add)
06 +2196 (store)
```

^① 译注：本书中文版配套资源在本章示例目录下的 `exercises\Simple` 子目录中提供了示例程序 `Simpletron.java`，详情请访问 <https://bookzhou.com>。它增强了练习 6.19～练习 6.20 中开发的基础版 `Simpletron.java`，支持将结果输出到文件（`output.txt`）。

^② 译注：本书中文版配套资源在本章示例目录下的 `exercises\Simple` 子目录中提供了示例程序 `InfixToPostfixConverter.java`，详情请访问 <https://bookzhou.com>。它增强了练习 12.22 中开发的同名程序，支持多位整数操作数和单字母变量名操作数。

07 +2096 (load)
08 +2198 (store)

前三个指令是将 1 加到 y 上的指令序列。指令 06 和 07 将累加器的值存入临时位置 96，然后又立即从该位置将值加载回累加器，以便指令 08 能将该值存入位置 98。通常，一个指令序列后面会紧跟着一条针对刚存储过的同一位置的加载指令。通过消除这对针对同一内存位置的 **store** 和 **load** 指令，就可以对代码进行优化。这种优化可以将 SML 程序的“内存占用”（memory footprint）减少 25%，并提高其执行速度。下表展示了图 12.17 程序的优化后 SML 代码，可以看到优化后的代码减少了 4 个指令。请修改你的编译器，以执行在本练习中学到的优化技术。

Simple 程序	SML 位置和指令	描述
5 rem 输入 x	无	rem 被忽略
10 input x	00 +1099	将 x 读入位置 99
15 rem 检查 y == x	无	rem 被忽略
20 if y == x goto 60	01 +2098 02 +3199 03 +4211	将 y（位置 98）加载到累加器 将 x（位置 99）从累加器上减去 为零则跳转到位置 11
25 rem 递增 y	无	rem 被忽略

30 let y = y + 1	04 +2098	将 y（位置 98）加载到累加器
	05 +3097	将 1（位置 97）加到累加器上
	06 +2198	将累加器存储到 y 中（位置 98）
35 rem 在 total 上累加 y	无	rem 被忽略
40 let t = t + y	07 +2096	将 t（位置 96）加载到累加器
	08 +3098	在累加器上加 y（位置 98）
	09 +2196	将累加器存储到 t 中（位置 96）
45 rem 重复 y == x 测试	无	rem 被忽略
50 goto 20	10 +4001	跳转到位置 01
55 rem 输出结果	无	rem 被忽略
60 print t	11 +1196	向屏幕输出 t（位置 96）
99 end	12 +4300	终止执行

12.27（增强 Simple 编译器）请对 Simple 编译器进行以下修改。部分修改需要相应地调整在练习 6.20 中编写的 Simpletron 虚拟机程序。其中许多任务都具有挑战性，可能需要投入大量时间和精力。^①

- 将 Simpletron 的内存修改为 1000 个单元（位置编号为 000~999），并修改编译器，生成适应 1000 单元 Simpletron 内存的机器语言。
- 允许编译器处理浮点数值及整数。Simpletron 虚拟机也必须进行相应修改以处理浮点数值。
- 增加对一元减号的支持，以表示负整数值。
- 允许在 let 语句中使用求余操作符（%）。修改 Simpletron 机器语言来包含求余（模除）

^① 译注：本书练习 12.26 与 12.27 的答案因时间所限，暂未由译者完成。我们诚挚地邀请您访问本书的 GitHub 项目（<https://github.com/transbot/JavaHowToProgram12e-Chinese>）并提交您的解答，共同完善本项目。另外，Simpletron 虚拟机的动画演示网址是 <https://simpletron.bookzhou.com/>。

指令。

- e) 允许在 `let` 语句中使用[^]作为求幂操作符。修改 Simpletron 机器语言来包含求幂指令。
- f) 允许编译器在 Simple 语句中区分大小写字母，因此，`x` 和 `X` 将被视为不同的变量。此项无需修改 Simpletron 虚拟机。
- g) 允许 `input` 语句读取多个变量的值，例如 `input x, y`。此项无需修改 Simpletron 虚拟机。
- h) 允许编译器在单个 `print` 语句中输出多个值，例如 `print a, b, c`，并用逗号和一个空格分隔输出的每个值。此项无需修改 Simpletron 虚拟机。
- i) 允许 `print` 语句的操作数是中缀表达式。
- j) 为编译器添加语法检查功能，使其在 Simple 程序中遇到语法错误时能输出错误消息。此项无需修改 Simpletron 虚拟机。
- k) 允许使用整型数组。此项无需修改 Simpletron 虚拟机。
- l) 允许通过 Simple 命令 `gosub` 和 `return` 指定子例程。`gosub` 命令将程序控制权传递给子例程，而 `return` 命令则将控制权交回 `gosub` 之后的语句。这类似于 Java 的方法调用，同一个子例程可以被程序中分布的多个 `gosub` 调用。此项无需修改 Simpletron 虚拟机。
- m) 允许使用以下形式的循环语句（每个代码行仍需添加行号）：

```
for x = 2 to 10 step 2
  rem Simple 语句
next
```

该 `for` 语句从 2 遍历到 10，默认增量为 1。`next` 行标记 `for` 语句循环体的结束。此项无需修改 Simpletron 虚拟机。

- n) 允许使用以下形式的循环语句（每个代码行仍需添加行号）：

```
for x = 2 to 10 step 2
  rem Simple 语句
next
```

该 `for` 语句从 2 遍历到 10，增量为 2。此项无需修改 Simpletron 虚拟机。